

LAPORAN UAS PRAKTIKUM

Sistem Rekomendasi

Fokus Permasalahan: *Cold-Start Problem* dan *Data Sparsity*

Solusi / Pendekatan: *Hybrid Recommender System* (Neural Collaborative Filtering dipadukan dengan ekstraksi fitur semantik teks menggunakan Model Pre-trained BERT)

1. Identifikasi Permasalahan

Berdasarkan studi kasus dari source code yang dilampirkan, tren permasalahan utama yang diselesaikan dalam sistem ini adalah **Cold-Start Problem** dan **Data Sparsity**.

- **Cold-Start Problem (Item):** Muncul ketika terdapat item (film) baru di dalam sistem yang belum memiliki interaksi (rating/ulasan) sama sekali dari pengguna. Metode *Collaborative Filtering* konvensional tidak akan dapat merekomendasikan item ini karena ketiadaan riwayat data interaksi.
- **Data Sparsity:** Pada dataset sistem rekomendasi yang besar, matriks interaksi antara pengguna dan item sering kali sangat kosong (sparse), di mana mayoritas pengguna hanya memberikan rating pada sebagian kecil dari total film yang ada. Hal ini menurunkan performa algoritma tradisional secara signifikan.

2. Solusi dan Pemilihan Metode

Untuk mengatasi masalah *Cold-Start* dan *Data Sparsity* secara bersamaan (*Knowledge Utilization*), solusi yang diimplementasikan adalah membangun model **Hybrid Recommender System** dengan mengintegrasikan representasi interaksi kolaboratif dan representasi konten (teks). Metodologi yang digunakan adalah:

- **Neural Collaborative Filtering (NCF):** Digunakan untuk menangkap pola interaksi nonlinear yang kompleks antara pengguna dan item melalui *Embedding Layers* dan *Multi-Layer Perceptron (MLP)*. Rating dipetakan menjadi klasifikasi biner, di mana rating ≥ 3.5 dianggap sebagai interaksi positif (skor 1).
- **Model NLP BERT:** Memanfaatkan *pre-trained model* all-MiniLM-L6-v2 (menggunakan library SentenceTransformer) untuk mengekstrak *text embeddings* dari metadata film

(gabungan judul dan genre). Fitur kaya semantik ini memungkinkan sistem untuk memodelkan kemiripan antar film meskipun suatu film belum pernah di-rating sebelumnya.

3. Arsitektur dan Implementasi Model

3.1 Preprocessing Data & Feature Engineering

Tahapan awal melibatkan penggabungan dataset *ratings.csv* dan *movies.csv*. Identitas pengguna dan item diubah menjadi representasi kontinu (*categorical encoding*). Selanjutnya, fitur teks disusun secara gabungan dari *Title* dan *Genres*, lalu diencode ke dalam matriks vektor berdimensi 384 menggunakan BERT.

3.2 Arsitektur Model Hibrida

Model ini mengombinasikan tiga jalur masukan (*input layers*):

1. **User Input:** Dikonversi melalui *Embedding Layer* berdimensi 50.
2. **Item Input:** Dikonversi melalui *Embedding Layer* berdimensi 50.
3. **BERT Text Input:** Matriks fitur teks (berdimensi 384) melewati sebuah *Dense Layer* terpisah untuk mereduksi dimensi ke ukuran 64.

Ketiga representasi fitur (CF + Content-based) tersebut lalu digabungkan menggunakan Concatenate dan diteruskan ke jaringan *Multi-Layer Perceptron (MLP)* yang berukuran [128, 64, 32] neuron dengan regulasi *Dropout* sebesar 20%. Lapisan keluaran memanfaatkan fungsi aktivasi *sigmoid* guna memprediksi nilai probabilitas preferensi atau kecenderungan interaksi pengguna.

3.3 Evaluasi Pelatihan

Model dikompilasi memakai *optimizer Adam* dan menghitung selisih *binary crossentropy*. Menggunakan porsi 80% data untuk berlatih dan 20% untuk validasi/uji, model mengonvergensi pola data dan berhasil mencapai performa yang menjanjikan. Hasil evaluasi pada set pengujian menunjukkan:

```
Test Loss: 0.7186
Test Accuracy: 0.7213 (72.13%)
```

4. Pembuatan API dan Antarmuka Pengguna

Setelah pelatihan, *weights* (.h5), *embeddings* (.npy), dan kamus pemetaan (mapping) disimpan dan diaplikasikan dalam lingkungan *deployment* skala kecil dengan arsitektur *microservice*.

4.1 FastAPI Endpoint (Backend)

Framework FastAPI diinisialisasi untuk melayani REST API melalui *endpoint* tunggal (`/recommend/{user_id}`). *Endpoint* tersebut memuat model dan memanfaatkan algoritma pencarian probabilitas tertinggi untuk menemukan Top-5 film rekomendasi. *Web server* Uvicorn dijalankan dalam latar belakang mesin untuk menampung lalu lintas permintaan (*requests*) pada `127.0.0.1:8000`.

4.2 Antarmuka CLI (Command Line Interface)

Aplikasi klien berformat baris perintah (CLI) diprogram dengan memanfaatkan library `requests`. Klien ini terus meminta masukan dari pengguna (berupa *User ID*), memanggil layanan API lokal, kemudian mencetak hasil JSON berupa rekomendasi film dan persentase skor prediksinya secara visual:

```
=====
🎬 APLIKASI SISTEM REKOMENDASI FILM (CLI) 🎬
=====
➡ Masukkan User ID (contoh: 0, 15, 100): 15
⌚ Mengambil data dari API untuk User 15...

✅ REKOMENDASI TOP-5 UNTUK USER 15:
1. Big Night (1996) (Skor Probabilitas: 1.0)
2. Bad Boys II (2003) (Skor Probabilitas: 1.0)
3. Radio Days (1987) (Skor Probabilitas: 1.0)
4. Eye of the Tiger (1986) (Skor Probabilitas: 1.0)
5. Enemy at the Gates (2001) (Skor Probabilitas: 1.0)
```

5. Kesimpulan

Implementasi proyek ini mengonfirmasi bahwa metode hibrida yang memadukan profil kolaboratif via **Neural Collaborative Filtering** dan ekstraksi konten berbasis bahasa dari **BERT** sangat andal untuk mengatasi *Data Sparsity* serta *Cold-Start Problem*. Sistem rekomendasi dapat menganalisis relasi semantik pada metadata (konten) apabila rekam jejak historis pada sebuah film masih kosong. Hasilnya disajikan secara optimal lewat REST API dengan performa *inferensi* yang efisien.